

CSCI 650 - Algorithms and Computability

Assignment 4

Solutions to the written questions on this assignment should be submitted via PDF to Canvas before October 13th at 11:59 pm. Implementations for question 1 should be submitted to INGINious by the same deadline. Make sure to justify your answers.

I highly encourage you to collaborate with one another. However, you must write up your own solutions independently. Feel free to communicate via Discord and to post questions on the appropriate forum in Canvas. Do not post solutions.

Have fun!

Problems

1. In this problem we will develop and analyze two heuristic approaches to solving the Traveling Salesman problem, one using nearest neighbors and the other using simulated annealing.

a. (5 pts) Write a function `totalDistance(D, path)` which returns the total distance a traveling salesman must travel given a sequence of cities. `path` will include each city exactly once, note that the salesman must end at the start city. Submit your code on INGINious and include a code snippet of this function in your PDF submission.

Answer.

The function adds each distance along the route. Then it includes the trip back home. That last edge matters. An empty route returns zero.

```
def totalDistance(D, path):
    if not path:
        return 0.0
    return float(sum(D[path[j]][path[(j + 1) % len(path)]]
                     for j in range(len(path))))
```

Each listed edge gets visited once. That takes $O(n)$ time and $O(1)$ extra space.

b. (10 pts) Write a function `nearestNeighbor(D, start)` which returns an order in which to visit cities with associated distance matrix `D` starting at `start` and following the nearest neighbor heuristic. Submit your code on INGINious and include a code snippet of this function in your PDF submission. What is the asymptotic runtime of this function?

Answer.

The route starts at the given city. A start of -1 picks one randomly. Each next stop is the nearest unvisited city. Ties go to the smaller city index.

```
def nearestNeighbor(D, start=-1):
    n = len(D)
```

```

if n == 0:
    return []
if start == -1:
    start = int(np.random.randint(n))
path = [start]
remaining = set(range(n))
remaining.remove(start)
while remaining:
    city = min(remaining, key=lambda v: (D[path[-1]][v], v))
    path.append(city)
    remaining.remove(city)
return path

```

The search checks $n-1$ cities, then $n-2$, and so on. These checks add up to $n(n-1)/2$. So the runtime is $\Theta(n^2)$. Matrix access and expected set operations take constant time. The path and remaining set use $O(n)$ extra space. The greedy choices can miss a better tour.

c. (5 pts) Use the `tspTests.py` script provided on Canvas to generate histograms depicting the distribution of ratios (observed distance)/(minimum observed distance) under different circumstances. Describe what you observe.

Answer.

I used the supplied tests with seed 65004. Both NumPy and Python's random seed were set. Each family has 50 matrices with 100 cities. Every starting city was tested on each matrix. That gives 5,000 ratios per family, using 25 bins.

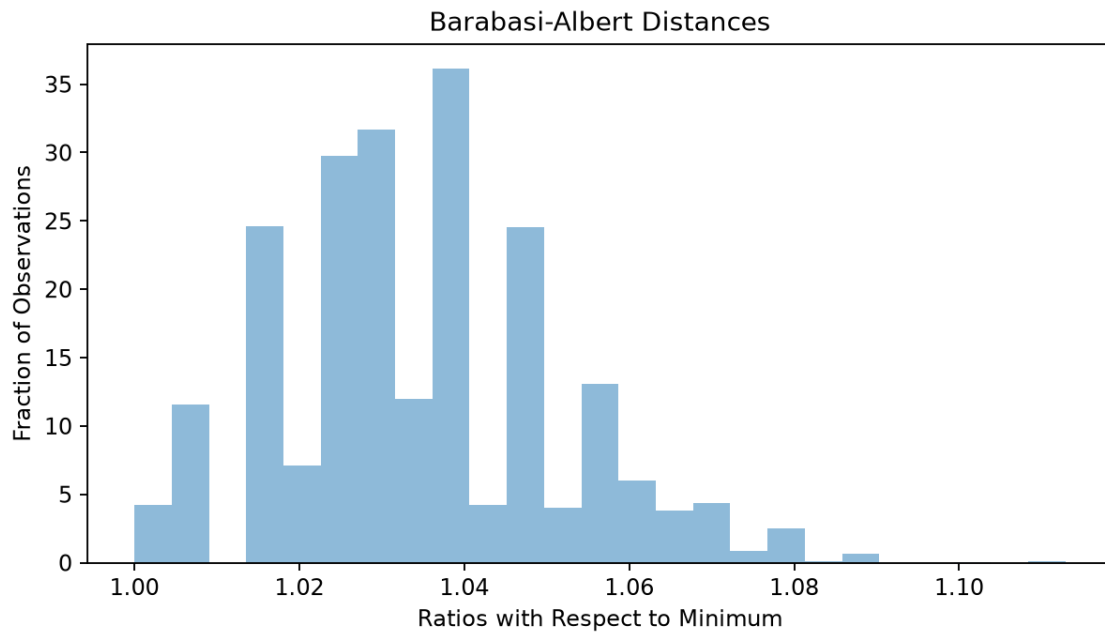
Each distance is divided by that matrix's best observed distance. This denominator comes from nearest-neighbor tours. It is not necessarily the true minimum.

Family	Mean	Median	95th percentile	Maximum
Barabasi-Albert	1.034	1.032	1.065	1.113
Watts-Strogatz	1.092	1.085	1.198	1.317
Uniform Random	1.421	1.411	1.772	2.188

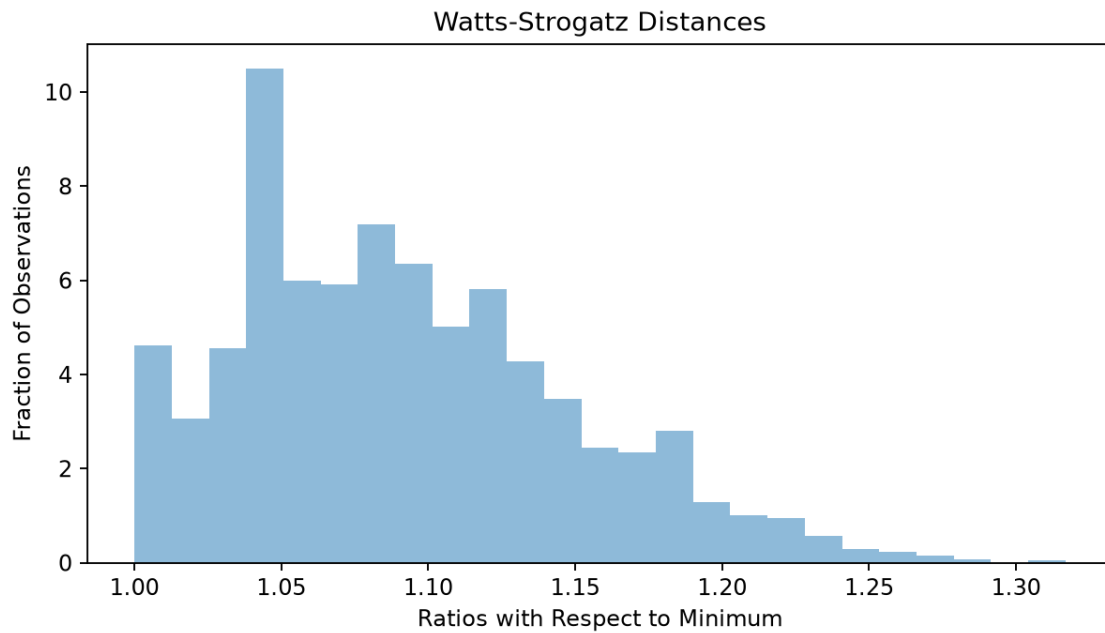
Barabasi-Albert stays fairly close to one. Its average gap is about 3.45%. Watts-Strogatz spreads out more, averaging about 9.20%. Uniform Random varies much more between starts. Its average gap is about 42.09%. The largest ratio reaches 2.188.

Uniform Random uses asymmetric distances without guaranteed triangle inequalities. A cheap early choice can leave an expensive return. Trying several starts seems useful here.

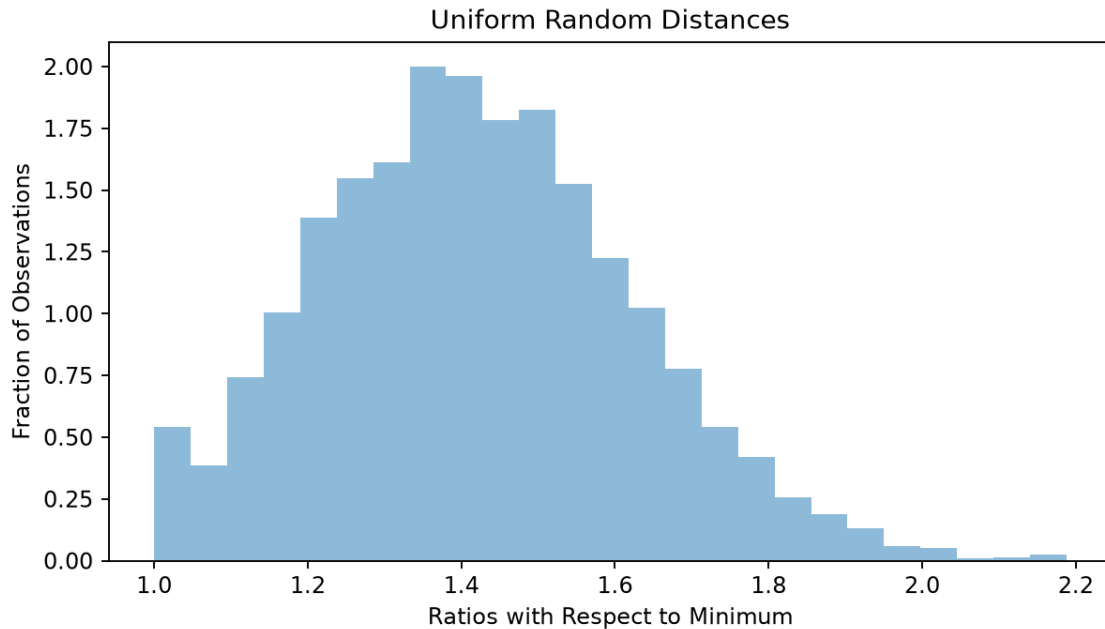
These observations are not independent. Starts on one matrix share the same denominator. These samples also cannot prove an approximation guarantee.



The horizontal axis shows the distance ratio. The vertical axis shows probability density.



The horizontal axis shows the distance ratio. The vertical axis shows probability density.



The horizontal axis shows the distance ratio. The vertical axis shows probability density.

d. (5 pts) Write three functions to generate candidate solutions. - `swapPairs(path, numSwaps, adj)` swaps `numSwaps` random pairs of elements in `path`. If `adj` is `True`, only swap adjacent pairs of elements in the current path. - `permuteRange(path)` randomly permutes a slice of `path` between two randomly selected indices. - `reverseRange(path)` reverses a slice of `path` between two randomly selected indices. Include code snippets of these functions in your PDF submission.

Answer.

Each candidate starts with a copy of the path. That leaves the input alone. Adjacent swaps use consecutive list positions, without wrapping around. Slices include both randomly selected endpoints. Paths shorter than two cities stay unchanged.

```
def swapPairs(path, numSwaps=1, adj=False):
    candidate = path[:] #we do not want to change the given path
    n = len(candidate)
    if n < 2:
        return candidate
    for _ in range(numSwaps):
        if adj:
            a = int(np.random.randint(n - 1))
            b = a + 1
        else:
            a, b = np.random.choice(n, size=2, replace=False)
            candidate[a], candidate[b] = candidate[b], candidate[a]
    return candidate

def permuteRange(path):
    candidate = path[:] #we do not want to change the given path
```

```

if len(candidate) < 2:
    return candidate
a, b = sorted(np.random.choice(len(candidate), size=2, replace=False))
candidate[a:b + 1] = np.random.permutation(candidate[a:b + 1]).tolist()
return candidate

def reverseRange(path):
    candidate = path[:] #we do not want to change the given path
    if len(candidate) < 2:
        return candidate
    a, b = sorted(np.random.choice(len(candidate), size=2, replace=False))
    candidate[a:b + 1] = candidate[a:b + 1][::-1]
    return candidate

```

Shuffling can leave a slice unchanged. Several swaps can also undo earlier swaps. Both still produce valid candidates.

With fixed swaps, each function takes $O(n)$ time. Copying the route already takes that long. NumPy's sampling may build a full permutation. For s swaps, a conservative bound is $O(n(1+s))$. Extra space is $O(n)$.

e. (10 pts) Write a function `runSimulatedAnnealing(D, genCandidate, maxIterations, T, alpha)` which finds a path for a traveling salesman using simulated annealing given a distance matrix D , a function to generate candidate solutions, a maximum number of iterations to perform, an initial temperature, and a multiplicative update factor for temperature. The initial path should be a random permutation of the cities. Include a code snippet of this function in your PDF submission. What is the asymptotic runtime of this function with respect to n (the number of cities) and i (the number of iterations)?

Answer.

Annealing begins with a random tour. Equal or shorter candidates are always accepted. Longer candidates get a chance too. Their acceptance probability is $e^{-\Delta/T}$ when $T > 0$. The temperature gets multiplied by α each iteration.

The best tour stays in a separate record. An accepted uphill move cannot erase that record. At zero temperature, uphill moves are rejected. The distance list records each iteration's current distance.

```

def runSimulatedAnnealing(D, genCandidate=swapPairs, maxIterations=1000, T=-1, alpha=0.9):
    if T==-1: T = np.sqrt(len(D)) #default starting temperature
    T = float(T)
    curPath = np.random.permutation(len(D)).tolist() #a random initial path
    curDist = totalDistance(D, curPath)
    bestPath, bestDist = curPath[:], curDist
    distList = []
    for _ in range(maxIterations):
        candidate = genCandidate(curPath[:])
        candidateDist = totalDistance(D, candidate)
        delta = candidateDist - curDist
        if delta <= 0 or (T > 0 and np.random.random() < np.exp(-delta / T)):
            curPath, curDist = list(candidate), candidateDist
            if curDist < bestDist:
                bestPath, bestDist = curPath[:], curDist
            distList.append(curDist)
        T *= alpha
    return bestPath, bestDist, distList

```

Each provided generator takes $O(n)$ time. Scoring and copying routes also take $O(n)$. For i iterations, the runtime is $O(n+in)$. Extra space is $O(n+i)$, including the history.

Those bounds assume at least one city. Including empty inputs gives $O((i+1)(n+1))$ time. For a general generator taking $G(n)$ time, the runtime becomes $O(n+i(n+G(n)))$.

f. (5 pts) Use the `tspTests.py` script provided on Canvas to generate plots showing how the total distance of the current solution changes over time. Discuss the results and briefly describe a process for tuning the hyperparameters of this approach.

Answer.

Each family used one 100-city matrix. Each candidate method ran for 10,000 iterations. The methods shared a matrix but had different starting tours. The settings were $T=10$ and $\alpha=0.9$.

Family	Generator	Initial	Best / final
Barabasi-Albert	swapPairs	239.000	132.000
Barabasi-Albert	permuteRange	233.000	175.000
Barabasi-Albert	reverseRange	236.000	112.000
Watts-Strogatz	swapPairs	494.000	176.000
Watts-Strogatz	permuteRange	499.000	274.000
Watts-Strogatz	reverseRange	529.000	121.000
Uniform Random	swapPairs	52.781	10.147
Uniform Random	permuteRange	51.351	19.597
Uniform Random	reverseRange	49.458	23.863

All nine runs improved their starting distances. Each finished at its best recorded distance. Reversal did best on both symmetric graph instances. Its distances were 112 and 121. Pair swaps won on Uniform Random, reaching 10.147.

Reversing a symmetric segment preserves its internal edge costs. Only the boundary costs can change. Asymmetric distances make reversal more disruptive. Internal arcs change direction too. A large permutation can disturb many good edges.

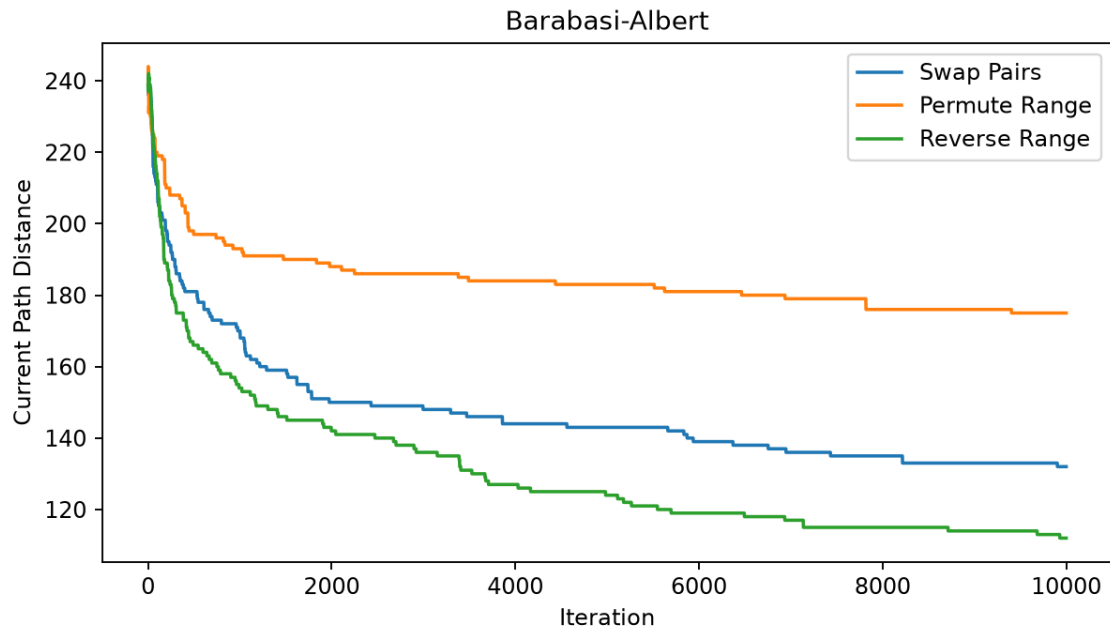
The current distance sometimes rises early. That comes from accepting uphill moves. Later, the curves mostly decrease in steps. A plateau does not prove optimality.

The default cooling gets cold pretty quickly. After 100 updates, T is about 0.000266. After 200, it is about 7.06×10^{-9} . Most remaining iterations behave like greedy improvement.

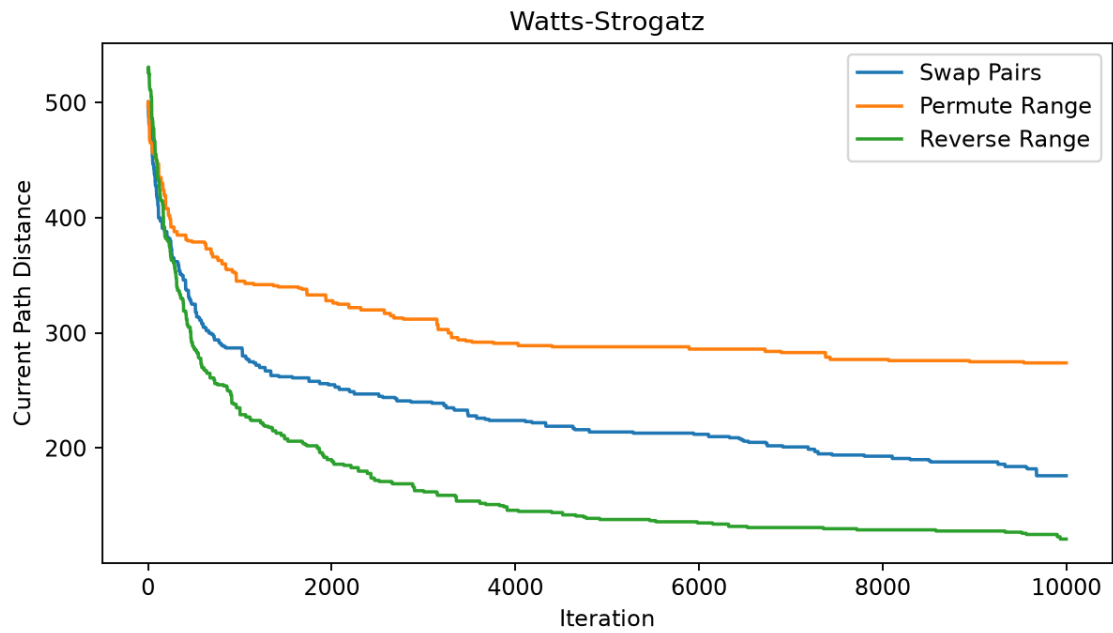
Sampling uphill changes helps choose a starting temperature. An initial acceptance probability of 0.8 is one option. A typical uphill change helps estimate that temperature.

I would use T_0 approximately $-\text{delta_typical}/\log(0.8)$. A final temperature sets the other endpoint. The schedule becomes $\alpha=(T_{\text{final}}/T_0)^{1/i}$. This spreads cooling across the iteration budget.

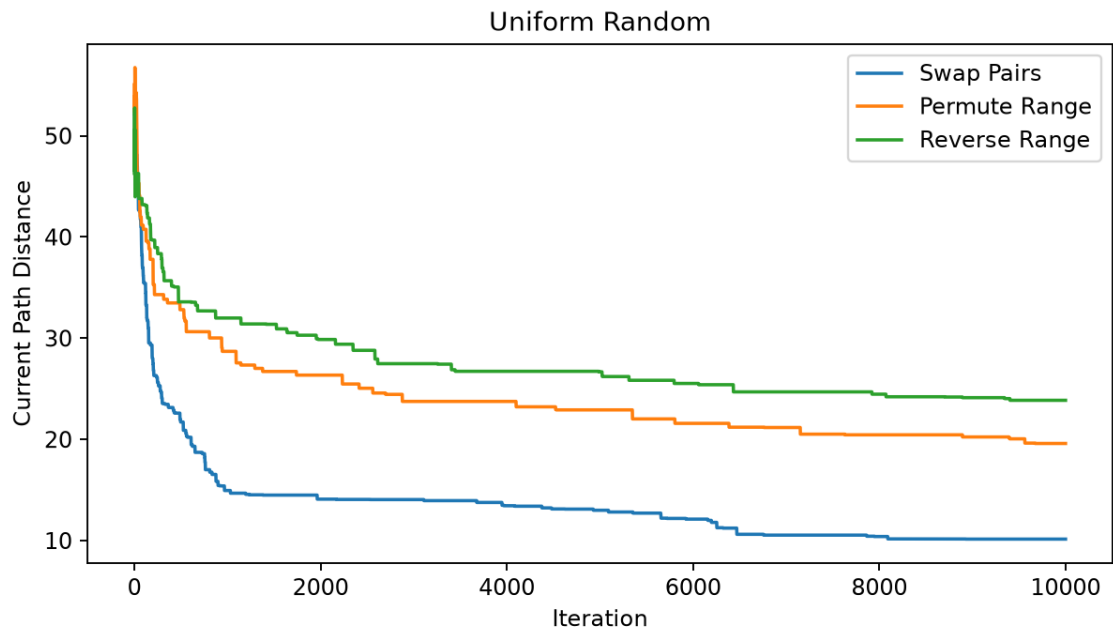
Several seeds should be compared under equal evaluation budgets. Sharing matrices and starting tours makes comparisons fairer. Fresh instances provide a final check. These single runs cannot establish a consistent winner.



The horizontal axis shows iterations. The vertical axis shows current tour distance.



The horizontal axis shows iterations. The vertical axis shows current tour distance.



The horizontal axis shows iterations. The vertical axis shows current tour distance.

2. Let us return to the busy beaver numbers.

a. (5 pts) Describe why the busy beaver numbers are incomputable intuitively. What are the practical consequences of this with respect to determining $BB(n)$?

Answer.

Here, $BB(n)$ uses the course's step-count definition. It measures the longest halting run among n -state machines. They use binary tapes, initially filled with zeros. The separate halt state does not count. This matches the earlier $BB(2)=6$ convention.

For fixed n , only finitely many machines exist. Each halting machine has a finite runtime. So the maximum halting time is finite.

The trouble is identifying every machine that never halts. Running one longer does not settle that question. It might still halt much later. A computable time bound would decide halting. That is impossible.

Halting examples provide lower bounds. Longer searches might improve those bounds. A cutoff cannot generally establish the exact maximum. Exact values need proofs covering the remaining machines. Some small values can be established. No single algorithm can compute every $BB(n)$.

b. (10 pts) Show that the busy beaver numbers are incomputable via reduction from a problem discussed in class. In particular, write pseudocode (using Python syntax) showing that a known undecidable problem can be solved assuming the existence of a function $BB(n)$ which is guaranteed to halt.

Answer.

Suppose $BB(n)$ is computable and always halts. That would let the halting problem be decided.

Start with a machine M and input w . Construct a blank-start, binary-tape machine N . It first writes an encoding of w . Then it simulates M on that input. This finite construction preserves whether the machine halts.

Count N 's working states, including the added setup states. Using only M 's original state count would be wrong.

```
def decideHalting(M, w):
    N = compileToBinaryBlankStart(M, w)
    n = numberOfWorkingStates(N)
    limit = BB(n)
    config = blankConfiguration(N)
    if isHalted(config):
        return True
    for _ in range(limit):
        config = step(N, config)
        if isHalted(config):
            return True
    return False
```

The initial check covers halting without any transitions. Each later check follows a simulated transition. That includes halting exactly at the bound.

If N halts, its runtime cannot exceed $BB(n)$. If it survives that bound, it cannot halt later. Otherwise it would exceed the maximum. The procedure therefore answers halting correctly.

The assumed BB call terminates. The bounded simulation also terminates. Together they would decide an undecidable problem. So a total computable BB function cannot exist.

This argument uses halting time, not printed-one counts. A bound on printed ones needs a different reduction. It cannot directly bound the number of steps.